Compiler Directives in Verilog HDL

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **3**

2 Basic Syntax **3**

3 The `define Directive **4**
3.1 Syntax . . . 4
3.2 Example: Defining a Constant . . . 4
3.3 Example: Expression Macro . . . 4

4 The `undef Directive **5**
4.1 Syntax . . . 5
4.2 Example . . . 5

5 The `include Directive **5**
5.1 Syntax . . . 5
5.2 Example . . . 5

6 Conditional Compilation **6**
6.1 The `ifdef Directive . . . 6
6.1.1 Syntax . . . 6
6.1.2 Example . . . 6
6.2 The `ifndef Directive . . . 6
6.2.1 Example . . . 6
6.3 The `else Directive . . . 7
6.3.1 Example . . . 7
6.4 The `elsif Directive . . . 7
6.4.1 Example . . . 7

7 Nested Conditional Compilation **8**
7.1 Example . . . 8

8 Include Guards **8**
8.1 Example: Header File . . . 8

9 The `timescale Directive **9**
9.1 Syntax . . . 9
9.2 Example . . . 9

10 The `default_nettype Directive **10**
10.1 Syntax 10
10.2 Example 10

11 The `resetall Directive **10**
11.1 Example 10

12 The `celldefine and `endcelldefine Directives **11**
12.1 Example 11

13 The `line Directive **11**
13.1 Example 11

14 Macro Definitions vs. Parameters **12**
14.1 Example Using a Parameter 12

15 Compiler Directives Summary **12**

16 Common Errors and Best Practices **13**

17 Summary **13**

1 Introduction

Compiler directives are special instructions in Verilog HDL that control how source code is processed before or during compilation and simulation setup.

They begin with a backtick character (`) and are used for macro definitions, conditional compilation, file inclusion, time unit specification, and other source-processing tasks.

Compiler directives are also commonly called preprocessor directives.

Unlike ordinary Verilog statements, directives do not generally describe hardware directly. Instead, they influence how the Verilog source is interpreted or processed.

Common compiler directives include:

- `define
- `undef
- `include
- `ifdef
- `ifndef
- `elsif
- `else
- `endif
- `timescale
- `default_nettype
- `resetall
- `celldefine
- `endcelldefine
- `line

2 Basic Syntax

A compiler directive starts with a backtick followed by the directive name.

```
// Define a macro
‘define WIDTH 8
// Use the macro
module example (
    input [‘WIDTH-1:0] a,
    output [‘WIDTH-1:0] y
);

    assign y = a;

endmodule
```

The macro `WIDTH` is replaced by its defined text during preprocessing.

Compiler directives are not terminated with semicolons unless a semicolon is part of the ordinary Verilog statement that follows them.

3 The `` `define `` Directive

The `` `define `` directive creates a textual macro.

A macro can represent a constant, an expression, or a reusable fragment of source code.

3.1 Syntax

```
`define MACRO_NAME replacement_text
```

3.2 Example: Defining a Constant

```
`define WIDTH 8

module define_example (
    input [`WIDTH-1:0] a,
    output [`WIDTH-1:0] y
);

    assign y = a;

endmodule
```

The macro `WIDTH` is replaced by `8` wherever it is used while the macro is defined.

3.3 Example: Expression Macro

```
`define ADD(a,b) ((a) + (b))

module macro_expression;

    reg [7:0] x, y;
    wire [8:0] sum;

    assign sum = `ADD(x,y);

endmodule
```

Parentheses around macro arguments and the complete expression help prevent operator precedence errors.

Macro expansion is textual. It does not provide the type checking or scoping of a Verilog function or parameter.

4 The `undef Directive

The `undef directive removes a previously defined macro.

4.1 Syntax

```
‘undef MACRO_NAME
```

4.2 Example

```
‘define WIDTH 8

module example_a;
    reg [‘WIDTH-1:0] data;
endmodule

‘undef WIDTH

// WIDTH is no longer defined here.
```

After `undef WIDTH, later source code cannot rely on that macro remaining defined.

This directive is useful when a macro should only be available within a limited portion of a source file.

5 The `include Directive

The `include directive inserts the contents of another source file into the current source at the location of the directive.

5.1 Syntax

```
‘include "filename.v"
```

5.2 Example

Suppose a file named definitions.vh contains:

```
‘define DATA_WIDTH 8
‘define RESET_VALUE 0
```

Another Verilog file can include it:

```
‘include "definitions.vh"
module include_example (
    input [‘DATA_WIDTH-1:0] data,
    output [‘DATA_WIDTH-1:0] result
);
    assign result = data;
endmodule
```

The included file is processed as though its contents were inserted at the inclusion point.

The compiler or simulator must be able to find the included file through its configured paths.

Files with the extension `.vh` are often used for Verilog header files, although this extension is a convention rather than a requirement.

6 Conditional Compilation

Conditional compilation includes or excludes source code according to whether a macro is defined.

It is commonly used to select simulation models, debug statements, feature configurations, or implementation alternatives.

6.1 The `` `ifdef `` Directive

The `` `ifdef `` directive checks whether a macro has been defined.

6.1.1 Syntax

```
`ifdef MACRO_NAME
    // Included when the macro is defined
`endif
```

6.1.2 Example

```
`define DEBUG

module debug_example;
    initial begin
`ifdef DEBUG
        $display("Debug mode enabled");
`endif
    end
endmodule
```

Because `DEBUG` is defined, the display statement is included in the source.

6.2 The `` `ifndef `` Directive

The `` `ifndef `` directive checks whether a macro has not been defined.

6.2.1 Example

```
`ifndef SYNTHESIS
    initial begin
        $display("Simulation-only message");
    end
`endif
```

The code is included only when `SYNTHESIS` is not defined.

The macro `SYNTHESIS` is commonly supplied by a tool flow; it is not guaranteed to be defined by every simulator or synthesis tool.

6.3 The `` `else `` Directive

The `` `else `` directive selects the alternative branch of a conditional compilation block.

6.3.1 Example

```
‘ifdef SIMULATION
    ‘define MODE_VALUE 1
‘else
    ‘define MODE_VALUE 0
‘endif
```

If `SIMULATION` is defined, the first definition is selected. Otherwise, the second definition is selected.

6.4 The `` `elsif `` Directive

The `` `elsif `` directive checks another macro when the preceding condition is false.

6.4.1 Example

```
‘ifdef FPGA
    // FPGA-specific implementation
‘elsif ASIC
    // ASIC-specific implementation
‘else
    // Default implementation
‘endif
```

Only the branch selected by the conditional compilation chain is included.

The first matching branch is selected when multiple listed macros are defined.

7 Nested Conditional Compilation

Conditional compilation directives can be nested to support multiple configuration options.

7.1 Example

```
`define SIMULATION
`define DEBUG

module nested_example;

`ifdef SIMULATION
    initial begin
`ifdef DEBUG
        $display("Simulation debug enabled");
`else
        $display("Simulation mode");
`endif
    end
`endif

endmodule
```

In this example, the debug message is included because both `SIMULATION` and `DEBUG` are defined.

Nested conditional directives must be balanced with matching `` `endif `` directives.

8 Include Guards

Include guards prevent the same header file from being processed repeatedly when it is included multiple times.

8.1 Example: Header File

The file `common.vh` can contain:

```
`ifndef COMMON_VH
`define COMMON_VH

`define DATA_WIDTH 8
`define ADDRESS_WIDTH 16

`endif
```

When the file is included again in the same macro-processing context, `COMMON_VH` is already defined, so its contents are skipped.

Include guards are useful for shared macro definitions and declarations.

9 The `timescale Directive

The `timescale directive specifies the time unit and time precision used for delay interpretation in a Verilog source region.

9.1 Syntax

```
‘timescale time_unit / time_precision
```

9.2 Example

```
‘timescale 1ns/1ps

module timescale_example;

    reg clk;

    initial begin
        clk = 0;
        #5 clk = 1;
        #5 clk = 0;
        #10 $finish;
    end

endmodule
```

With this timescale, #5 represents a delay of 5 nanoseconds.

The time precision is 1 picosecond. It determines the resolution to which delays are rounded.

Use consistent time settings across source files to make simulation results easier to understand.

10 The `default_nettype Directive

The `default_nettype directive controls the default net type used for implicitly declared nets.

10.1 Syntax

```
‘default_nettype none
```

10.2 Example

```
‘default_nettype none

module nettype_example (
    input a,
    output y
);

    assign y = a;

endmodule

‘default_nettype wire
```

With `default_nettype none, an undeclared net cannot be created implicitly.

This helps catch spelling mistakes in signal names and prevents accidental implicit nets.

The directive remains in effect until another `default_nettype directive changes it.

It is good practice to restore the intended default at the end of a source file when required by the surrounding compilation flow.

11 The `resetall Directive

The `resetall directive resets compiler directive settings to their default values.

It can reset settings such as the active timescale, net type, and other directive-controlled compilation settings.

11.1 Example

```
‘timescale 1ns/1ps
module resetall_example;
    initial begin
        #10;
        $display("Simulation time = %0t", $time);
    end
endmodule
‘resetall
```

This directive can help prevent certain compiler directive settings from unintentionally affecting subsequent source text.

12 The `celldefine and `endcelldefine Directives

These directives mark module definitions as cell modules for tools that recognize cell information.

12.1 Example

```
‘celldefine

module simple_cell (a, y);

    input a;
    output y;

    assign y = ~a;

endmodule

‘endcelldefine
```

These directives are mainly relevant to simulation and implementation flows that recognize cell definitions.

Their practical effect depends on the tool and its configuration.

13 The `line Directive

The `line directive supplies source file and line-number information to tools for diagnostics and debugging.

13.1 Example

```
‘line 100 "design.v" 0

module line_example;
    initial begin
        $display("Line directive example");
    end
endmodule
```

This directive is primarily useful in generated or preprocessed source code. Tool support may vary.

14 Macro Definitions vs. Parameters

Macros and parameters can both represent constant values, but they work differently.

Feature	Macro (`define)	Parameter
Processing	Textual substitution.	Elaboration-time constant.
Scope	Preprocessor context.	Associated with a module or other supported design construct.
Typing	No inherent type checking.	Can have a declared type in supported language versions.
Override	Can be changed using macro directives or tool-defined macro settings.	Can be overridden through supported parameter override mechanisms.
Typical usage	Feature switches and shared text.	Configurable hardware dimensions and values.

14.1 Example Using a Parameter

```
module parameter_example #(
    parameter WIDTH = 8
) (
    input [WIDTH-1:0] a,
    output [WIDTH-1:0] y
);
    assign y = a;
endmodule
```

A parameter is usually preferable when defining a module's configurable width or another elaboration-time hardware property.

15 Compiler Directives Summary

Directive	Purpose
`define	Defines a textual macro.
`undef	Removes a previously defined macro.
`include	Includes another source file.
`ifdef	Includes code when a macro is defined.
`ifndef	Includes code when a macro is not defined.
`elsif	Checks an additional conditional branch.
`else	Selects the alternative conditional branch.
`endif	Ends a conditional compilation block.
`timescale	Sets time unit and time precision for delays.
`default_nettype	Controls the default implicit net type.
`resetall	Resets compiler directive settings to defaults.
`celldefine	Marks following module definitions as cell modules.
`endcelldefine	Ends the cell-definition region.
`line	Supplies source line information to tools.

16 Common Errors and Best Practices

Common mistakes include:

- Forgetting to terminate a conditional compilation block with `` `endif ``.
- Including a file that cannot be found in the configured search paths.
- Defining macros with names that conflict with other files or libraries.
- Assuming macros have the same scope and type checking as parameters.
- Omitting parentheses in expression macros.
- Using inconsistent timescale settings across source files.
- Allowing implicit nets when strict declaration checking is required.
- Assuming every simulator or synthesis tool supports every directive identically.

Recommended practices include:

- Use descriptive and unique macro names.
- Use include guards in shared header files.
- Prefer parameters for configurable hardware.
- Use `` `default_nettype none `` to detect undeclared nets where appropriate.
- Keep simulation-only code separate from synthesizable RTL.
- Check compiler and simulator options when using conditional compilation.

17 Summary

- Compiler directives begin with a backtick character.
- `` `define `` and `` `undef `` control macro definitions.
- `` `include `` inserts another source file.
- Conditional directives select which source code is included.
- `` `timescale `` defines time unit and precision for delay interpretation.
- `` `default_nettype none `` helps detect undeclared nets.
- Include guards prevent repeated processing of shared header contents.
- Parameters are usually preferable to macros for configurable hardware dimensions.

Key takeaway: Compiler directives control source processing, conditional compilation, and simulation settings. Using them consistently improves code reuse, portability, and debugging in Verilog HDL projects.